
LLD – DAY 3

Inheritance & Polymorphism

◇ Access Modifiers (VERY IMPORTANT)

Access modifiers decide karte hain ki **class ke members (variables/methods)** kaha se access ho sakte hain.

☑ Types of Access Modifiers

Modifier	Accessibility
public	Kahi se bhi access
private	Sirf same class ke andar
protected	Same class + child (inherited) class

📌 Important Rule:

Private members parent ke bhi child class me access nahi hote

◇ Inheritance

☑ Definition

Inheritance is a mechanism where a **child class acquires properties and behaviors of a parent class**, and can also add its own specific features.

📌 Purpose:

- Code reusability
 - Logical hierarchy
 - Real-world modeling
-

🧠 Real-World Example

- Parent: **Car**
- Child:
 - ManualCar (gear system)
 - ElectricCar (battery system)

All cars:

- Brand
- Model

- Start
- Stop
- Accelerate

Specific cars:

- Manual → Gear
- Electric → Battery

Key Points of Inheritance

- Child class can access:
 - Public members
 - Protected members
- Child cannot access:
 - Private members
- Avoids code duplication
- Supports extensibility

◇ Inheritance Syntax (C++)

```
class ManualCar : public Car { };
class ElectricCar : public Car { };
```

 **public inheritance** maintains access levels.

Access Specifiers in Inheritance

Parent Member	public inheritance	protected inheritance	private inheritance
public	public	protected	private
protected	protected	protected	private
private	✗ Not inherited	✗	✗

Inheritance in LLD

- Represents **IS-A relationship**
- Helps build **hierarchical models**
- Enables reuse of common behavior

Example:

ManualCar IS-A Car

◇ Polymorphism

☑ Definition

Polymorphism means **one interface, many implementations**.

🔗 Derived from:

- **Poly** → Many
- **Morph** → Forms

🧠 Real-Life Meaning

Same action → different response

Example:

- ManualCar accelerates using gear
- ElectricCar accelerates using battery

◇ Types of Polymorphism

1 Static Polymorphism (Compile-Time)

2 Dynamic Polymorphism (Run-Time)

◇ 1. Static Polymorphism (Method Overloading)

☑ Definition

Same method name but **different parameter list**.

🔑 Key Points

- Decided at **compile time**
- Achieved using **method overloading**
- Improves readability

🔗 Rules

- Method name: same
- Parameters: different (number/type)
- Return type alone cannot differentiate

🧩 Example

```
void accelerate();  
void accelerate(int speed);
```

💡 Static Polymorphism in LLD

- Same operation with different inputs
- Cleaner APIs
- User flexibility

◇ 2. Dynamic Polymorphism (Method Overriding)

☑ Definition

Same method signature, **different implementation in child classes**, resolved at runtime.

🔑 Key Points

- Achieved using:
 - `virtual` keyword
 - Parent class pointer
- Supports **runtime behavior change**
- Very important for LLD

📌 Rules

- Same method name
- Same parameters
- Parent method must be `virtual`

🔗 Example

```
class Car {  
    virtual void accelerate() = 0;  
};
```

Child classes implement their own logic.

💡 Dynamic Polymorphism in LLD

- Loose coupling
- Easy extensibility
- Plug-and-play design

◇ Static vs Dynamic Polymorphism (VERY IMPORTANT)

Feature	Static	Dynamic
Binding time	Compile-time	Runtime
Achieved by	Overloading	Overriding
Keyword	✗	virtual
Flexibility	Low	High
LLD usage	Limited	Heavy

◇ Combined Use of OOPS Pillars

Your final system uses:

- **Abstraction** → Interface (Car)
- **Encapsulation** → Private/Protected members
- **Inheritance** → ManualCar, ElectricCar
- **Polymorphism** → accelerate(), brake()

📌 This is ideal LLD design

📄 Day 3 Summary (Revision Sheet)

- ✓ Inheritance = code reuse
 - ✓ Polymorphism = multiple behaviors
 - ✓ Static = compile-time
 - ✓ Dynamic = runtime
 - ✓ Protected = child access
 - ✓ Private = class only
-

🎯 Golden Interview Lines

- **Inheritance models IS-A relationship**
 - **Polymorphism enables runtime flexibility**
 - **Dynamic polymorphism is core of LLD**
 - **Protected is designed for inheritance**
-

